

SMI3.0.5 – Technical Validation Report

DRAFT – CODE-BASED REVIEW

Following the structure and validation-report style of the SMI2.0v3.5 Technical Validation Report

This draft is grounded exclusively in the SMI3 source code and configuration files provided for review. Statements requiring independent execution, production evidence, business documentation, data-source certification, or validation results are clearly identified as pending and must not be treated as completed validation conclusions.

1 Executive Summary

1.1 Preamble

SMI3.0.5 is implemented as a mortgage-insurance risk modelling framework that combines policy and borrower preprocessing, binary machine-learning prediction, external LGD integration, loss calculations, model evaluation, model registration, and real-time inference. The reviewed source code uses AWS SageMaker for orchestration, Snowflake and S3-based inputs for model data, MLflow for experiment/artifact tracking, and AutoGluon with XGBoost as the configured model family. The inference version is identified in code as SMI 3.0.5.

The technical validation objective is to assess whether the implementation is technically sound, reproducible, appropriately controlled, consistent with the intended model design, and suitable for independent replication. This review emphasizes data processing, feature engineering, target construction, model methodology, model diagnostics, implementation controls, explainability, MLOps, and governance.

The review is intentionally separated into (a) code-evidenced conclusions and (b) validation procedures that remain to be executed. Source code can demonstrate that a control or test is implemented, but cannot by itself establish that the control operated successfully on the final approved dataset or that model performance is acceptable.

1.2 SMI3.0.5 Key Design / Updates

- Automated preprocessing of policy and borrower data, with policy-level and borrower-level processors and application-level merging.
- Use of Snowflake for policy/borrower source data and S3 for static economic, claim/delinquency, geography, and credit-score inputs.
- Application-level borrower aggregation with multiple summary statistics and credit-score source selection logic.
- Binary target construction from mortgage performance categories including delinquency and claim events.
- Training population based on funded mortgages from BOOKYEAR 2009–2015 and a materially later test population based on EFFECT_SID 202101–202312.
- Macroeconomic feature construction using HPI and unemployment information.
- External LGD model dependency and downstream loss/loss-ratio calculations.
- AutoGluon binary classification restricted by configuration to XGBoost, with ROC AUC as the configured evaluation metric.
- Five-fold bagging configuration, fixed random seed (42), feature importance, and SHAP explainability.
- Evaluation outputs including predictions, model metadata, risk ranking, lift/gain, and category-capture artifacts.
- SageMaker pipeline orchestration covering preprocessing, quality/evaluation controls, training, evaluation, and model registration.

- Real-time inference code with JSON field validation, preprocessing, model loading, scoring, and versioned response output.

1.3 Technical Validation Scope

1.3.1 In Scope

- Review of data loading, joins, deduplication, missing-value treatment, and application-level aggregation.
- Review of training, testing, and recent population definitions and time-period filters.
- Review of target definition and event-category logic.
- Review of feature transformations, macroeconomic features, credit-score logic, and LGD integration.
- Review of feature exclusion and leakage-prevention controls.
- Review of AutoGluon/XGBoost model configuration, cross-validation/bagging, random seed, and model artifacts.
- Review of feature importance and SHAP implementation.
- Review of evaluation logic, risk-ranking utilities, and model-registration conditions.
- Review of inference validation and batch-versus-real-time implementation consistency.
- Review of logging, MLflow tracking, SageMaker orchestration, and code-level reproducibility controls.
- Definition of independent replication procedures and preliminary validation findings.

1.3.2 Out of Scope

- Certification of the accuracy of source-system data supplied through Snowflake, S3, or external files.
- Validation of the external LGD model methodology unless separate LGD documentation and validation evidence are supplied.
- Business approval of target definition, population selection, cut-offs, economic assumptions, or product treatment.
- Final privacy, legal, ethical, fairness, and third-party data-use conclusions.
- Final performance, calibration, stability, and fit-for-use conclusions until independent execution is completed.
- Production monitoring effectiveness until production monitoring evidence and observed results are supplied.
- Final model risk rating and residual-risk acceptance.

1.4 Technical Validation Summary

1. The submitted code shows a structured separation between policy processing, borrower processing, model training, evaluation, pipeline orchestration, and inference. This modularity supports traceability and independent review.
2. Policy and borrower data are combined at APPLICATION_NUMBER with a one-to-one merge validation. Duplicate handling and application-level claim aggregation are explicitly implemented. These are positive data-integrity controls that should be independently reconciled to source data.
3. The final training population is restricted to funded mortgages with BOOKYEAR 2009–2015 and non-missing credit score. The test population is based on EFFECT_SID 202101–202312. This creates a strong temporal separation for out-of-time assessment, but also creates a representativeness question that must be quantified.
4. The target is generated from performance categories. No_Event is assigned to the non-event class, event categories are generally assigned to the event class, and Others is treated as missing. The exact approved business interpretation and performance horizon require formal confirmation.

5. The code explicitly identifies leaky identifiers and temporal/metadata fields and removes analysis-only columns before training. Validation should independently inspect the final feature matrix to verify that no post-outcome or future-information variables remain.
6. The reviewed configuration specifies binary classification, ROC AUC, XGBoost-only training through AutoGluon, a fixed seed of 42, and five bagging folds. HPO is disabled in the submitted configuration. The model-selection and hyperparameter rationale therefore requires documented challenge.
7. Feature importance and SHAP outputs are implemented. These support interpretability, but final validation should assess economic reasonableness, stability, and segment consistency rather than accepting generated explainability artifacts at face value.
8. The evaluation framework creates predictions and risk-ranking artifacts. Independent replication is required before any quantitative conclusion is made regarding discrimination, calibration, lift, stability, or acceptance thresholds.
9. The SageMaker pipeline contains processing, training, evaluation, and model-registration logic. Validation should confirm that the exact validated code, configuration, image, and model artifact are the objects promoted through QA and production.
10. Based on code review, SMI3 contains several sound technical controls. A final fit-for-use conclusion remains conditional on independent execution, data and population validation, model performance replication, LGD dependency review, deployment reconciliation, and closure/acceptance of validation findings.

1.5 Model Fit for Use

Preliminary technical assessment: **CONDITIONALLY SUPPORTABLE**, subject to completion of independent validation. The reviewed code is sufficiently structured to support controlled validation and contains explicit controls for data processing, leakage prevention, model training, explainability, evaluation, and deployment. However, code review alone is not sufficient to conclude that the model is fit for production use.

A final “Fit for Use” recommendation should require: successful independent reproduction of preprocessing and model outputs; acceptable out-of-time discrimination and calibration; stable performance across material portfolio segments; confirmation of target and population design; review of the external LGD dependency; evidence of immutable code/configuration promotion; production inference parity; monitoring thresholds; and closure or formal acceptance of material findings.

2 Technical Review of SMI3.0.5

2.1 Data, Target and Model Development Methodology

2.1.1 Data / Component Separation and Integration

The SMI3 implementation separates policy-level and borrower-level transformations before joining the processed datasets by APPLICATION_NUMBER. Policy processing incorporates policy attributes, economic data, geographic/static data, and claim/delinquency information. Borrower processing incorporates borrower information and multiple credit-score sources. The resulting datasets are merged using a one-to-one validation constraint.

Validation assessment: The separation is methodologically reasonable because policy and borrower features have different source structures and aggregation requirements. The one-to-one merge validation is a useful control against accidental row multiplication. Independent validation should reconcile row counts before and after each major join and inspect unmatched applications, duplicate resolution, and loss of records.

Required validation evidence:

- Source-to-model data lineage showing each raw source, extraction date, owner, key fields, and transformation path.
- Row-count reconciliation from raw policy and borrower inputs to final model populations.
- Duplicate application analysis and justification for keeping the last policy record where duplicates occur.
- Unmatched-key analysis for policy-to-borrower merge.
- Reconciliation of claim/delinquency aggregation to raw episodes.
- Confirmation that static S3 files and Snowflake tables correspond to the approved model data cut.

2.1.2 Training Population, Feature and Model Fit Review

The preprocessing code ultimately restricts the training population to funded applications with non-missing credit score and BOOKYEAR between 2009 and 2015. A test population is created from EFFECT_SID between January 2021 and December 2023. The code also creates a recent population. This temporal separation is useful for independent out-of-time testing, but the long gap between development and testing increases the importance of representativeness and drift analysis.

The reviewed model configuration uses AutoGluon for a binary problem, restricts allowed models to XGBoost, uses ROC AUC as the evaluation metric, sets random_seed=42, uses five bagging folds, one bagging set, no stacking, no weighted ensemble, and zero HPO trials.

Validation challenge: The code supports a technically coherent training workflow, but the evidence supplied does not establish why the 2009–2015 development period is optimal for current underwriting, why later vintages are excluded from development, or why XGBoost-only with HPO disabled is preferable to reasonable alternatives.

2.1.2.1 Data and Model Diagnostics

- Profile event/non-event counts and rates by year, region, product, LTV, credit score, and major underwriting segments.
- Compare missingness and outlier rates between training, test, recent, and current production populations.
- Compare feature distributions using PSI/CSI or other agreed drift statistics.
- Verify that all training features are available at the actual scoring decision point.
- Reproduce five-fold out-of-fold predictions and compare OOF versus training-fit performance.
- Reproduce ROC AUC, KS/Gini if used, precision/recall, lift, and capture rates.
- Review calibration overall and by risk band.
- Inspect feature importance and SHAP dependence for major drivers.
- Investigate performance concentration in small segments or rare categories.

2.1.2.2 Distribution, Calibration and Ranking Diagnostics

The SMI2 report used distribution-fit and QQ diagnostics for distress. For SMI3, the analogous validation focus is the distribution of predicted probabilities, observed event rates, and rank ordering. Validation should compare predicted-score distributions across training, OOT test, and recent data; produce calibration plots; and assess whether higher predicted-risk bands consistently exhibit higher observed event rates.

Recommended exhibits include probability histograms by dataset, calibration curves, observed-versus-predicted event rates by decile/ventile, lift and cumulative-gain curves, KS plots, score distribution by event class, and segment-level calibration tables.

2.1.2.3 Conservatism, Bias and Loss Implications

SMI2 explicitly assessed loss conservatism. For SMI3, conservatism should be assessed at both probability and loss levels. Validation should determine whether model probabilities systematically over-

or under-predict observed events and whether the combination of probability and LGD produces conservative or aggressive loss estimates.

The assessment should distinguish statistical bias from deliberate business conservatism. Any post-model adjustment, floor, cap, multiplier, or calibration should be documented separately from the underlying model so that the technical integrity of the model can be evaluated independently.

2.1.3 LGD / Loss-Ratio Integration

SMI3 preprocessing references an external LGD model path, and downstream code combines probability and LGD-related information to produce loss outputs. The technical validation of SMI3 should therefore treat LGD as a material model dependency.

- Confirm the exact LGD model name, version, approval status, and intended use.
- Reconcile LGD inputs and outputs for a sample of applications.
- Verify that the LGD artifact used in training/evaluation is the same approved artifact used in inference.
- Assess sensitivity of final loss outputs to LGD changes.
- Confirm treatment of missing or invalid LGD values.
- Cross-reference the independent validation of the LGD model rather than duplicating it where appropriate.

2.1.4 Probability and LGD Contribution to Loss

Validation should decompose loss outputs into probability-driven and LGD-driven components. This is analogous to the SMI2 assessment of Gamma versus top-up loss contribution. The objective is to identify which component drives total loss variation and where model risk is concentrated.

Recommended analysis: calculate total expected loss under base outputs; hold LGD fixed and vary probability; hold probability fixed and vary LGD; summarize contribution by region, product, risk band, and vintage; and identify segments where a small probability change produces a material loss change.

2.1.5 Product / Regulatory Feature Treatment

The SMI2 report explicitly reviewed VRM/ARM multipliers associated with regulatory changes. The submitted SMI3 code contains product and underwriting fields but does not, from code evidence alone, establish an equivalent specific VRM/ARM calibration. Accordingly, this report does not infer one. Validation should identify any SMI3 product-specific rules, regulatory transformations, cut-offs, or post-model adjustments from approved business documentation and reconcile them to code. Where none exist, the section should state “not applicable” in the final report.

2.1.6 HPI and Unemployment Sensitivity

SMI3 explicitly constructs economic features involving HPI and unemployment. The test population applies regional economic values using a December 2022 reference, while recent data use a later reference determined in preprocessing. These choices require technical and business validation because macroeconomic timing can materially influence model behaviour.

- Verify economic source files and regional mapping.
- Confirm units and formulas for HPI and unemployment transformations.
- Assess whether the reference-date convention is consistent with intended scoring use.
- Demonstrate that no future economic information is used relative to the scoring date unless explicitly intended for scenario analysis.
- Run one-at-a-time sensitivity tests for HPI and unemployment.
- Run joint stress scenarios and confirm directionality is economically reasonable.
- Assess whether economic sensitivity is stable across regions and credit-risk bands.

2.2 SMI3 Model Diagnostics

2.2.1 Out-of-Time / Backtesting

The 2021–2023 test population provides a natural OOT dataset relative to the 2009–2015 training population. Independent validation should reproduce all developer OOT results from raw approved inputs and should not rely only on stored developer predictions.

- Reproduce model scores for the full OOT population.
- Compare predicted event rates with observed event rates overall and by period.
- Assess ROC AUC and rank ordering by year and material segment.
- Assess calibration by decile and risk category.
- Calculate lift/capture at operational cut-offs.
- Compare model loss estimates with realized loss where outcome maturity is sufficient.
- Investigate material underprediction or overprediction by province, product, lender/channel, LTV, and credit-score bands.

2.2.2 Model Training / Convergence / Reproducibility

XGBoost is an iterative boosting algorithm rather than a Monte Carlo simulation. The relevant SMI3 convergence review should therefore focus on training stability, fold consistency, reproducibility, and sensitivity to training configuration.

- Compare fold-level validation metrics and identify unstable folds.
- Review learning behaviour and early-stopping/training metadata where available.
- Repeat training using the same seed and immutable environment to confirm reproducibility.
- Repeat training under alternate reasonable seeds to assess variability.
- Compare model feature importance and SHAP ranking across repeated fits.
- Confirm that the model artifact produced by validation is materially identical in performance to the developer artifact.

2.2.3 Model Stability – Different Out-of-Time Data

The code creates test and recent populations, enabling stability assessment across different time windows. Validation should assess whether discrimination, calibration, and feature relationships remain stable as the portfolio and economic environment change. At minimum, report performance by calendar year, booking vintage, region, product, LTV band, and credit-score band. Where outcome maturity differs by period, distinguish fully matured from partially matured cohorts.

2.2.4 Model Sensitivity

Sensitivity testing should determine whether model outputs respond appropriately to changes in important inputs and whether small data perturbations create disproportionate output changes.

2.2.4.1 Random Seed / Training Sensitivity

The submitted configuration fixes `random_seed=42`. Validation should retrain the model under a small set of alternate seeds using the same approved data and configuration. Compare ROC AUC, calibration, risk ranking, top features, SHAP directionality, and application-level score movement. Material instability would indicate sensitivity to sample partitioning or stochastic training.

2.2.4.2 Challenger / Simpler Models

The SMI2 report reviewed lower-grade models. For SMI3, the analogous challenge is to compare the selected XGBoost implementation with simpler or alternative models. A logistic-regression benchmark is particularly useful for assessing incremental value, interpretability, and potential overfitting.

- Logistic regression benchmark using the same target and population.
- Simple decision tree or regularized generalized linear model where appropriate.
- Alternative gradient-boosting configuration or model family if permitted by governance.
- Comparison of OOT AUC, calibration, stability, and interpretability.
- Assessment of whether complexity provides a material and stable benefit.

2.3 Model Documentation

Final documentation should be complete enough for an independent reviewer to reproduce the model without relying on developer knowledge. The documentation should describe the business objective, intended use, limitations, target, performance horizon, data sources, development sample, exclusions, transformations, feature list, model methodology, hyperparameters, LGD dependency, evaluation metrics, explainability, implementation, monitoring, and change controls.

The final validation should perform a line-by-line reconciliation between key documentation claims and code/configuration. Any difference between documentation and implementation should be recorded as a finding or resolved before approval.

3 Independent Code Review

3.1 Independent Code Replication Environment

The independent validation environment should be controlled by the validation team and should use the validated Git commit, approved data snapshot, immutable Docker image, approved external LGD artifact, and the same configuration used for the candidate production model. Validation-owned outputs should be stored separately from developer artifacts.

Minimum replication evidence should include Git commit hash, image tag and digest, package/dependency manifest, configuration checksum, input-data identifiers, model artifact checksum, execution logs, and output checksums.

3.2 GitLab – Locked-Down Code

The SMI2 template explicitly requires locked-down GitLab code. For SMI3, final validation should confirm that the reviewed code is version controlled, peer reviewed, and protected from unapproved modification. The exact repository/branch controls are not established by the source files alone and require CI/CD evidence.

- Protected branch and merge-request controls.
- Commit hash corresponding to the validated release.
- Code-owner / reviewer approvals.
- CI test/build evidence.
- Container image provenance and immutable digest.
- Promotion evidence from development to QA/validation and production.
- Separation of duties between model development, validation, and production deployment.

3.3 Incremental Code Updates

For future SMI3 releases, technical validation should use a risk-based incremental review similar to SMI2. Each release should identify changed files, changed configuration, data-source changes, model-artifact changes, infrastructure changes, and expected output impact. Validation should independently inspect the diff and determine whether full or targeted revalidation is required.

For the current reviewed package, key code areas include preprocessing, policy processing, borrower processing, training, evaluation, pipeline orchestration, JSON validation, inference, loss calculation, and shared utilities.

3.4 Code Review Observations

Positive controls observed:

- One-to-one merge validation when combining policy and borrower datasets.

- Duplicate application and claim-episode handling explicitly logged and processed.
- Leaky and analysis-only columns explicitly excluded from model inputs.
- Fixed random seeds and MLflow logging support reproducibility.
- Feature importance, SHAP, risk-ranking, and validation artifacts are generated.

Validation challenges identified:

- Training population limited to 2009–2015 funded business; representativeness versus current business requires evidence.
- 2020 claim episodes excluded in policy processing; rationale and impact should be quantified.
- Test economic variables standardized to a December 2022 regional reference; rationale, leakage implications, and intended-use consistency should be confirmed.
- HPO disabled in the submitted configuration; model-selection and hyperparameter rationale should be documented.
- Dependency versions generally lower-bounded rather than fully pinned; immutable build controls should be evidenced.

4 Independent Result Replications

Independent result replication is a mandatory completion item for the final report. The following tests should be performed using validation-controlled execution:

1. Reproduce raw-to-processed row counts for policy, borrower, merged, training, test, and recent datasets.
2. Reproduce duplicate handling, claim aggregation, and borrower aggregation for sampled applications.
3. Reproduce target/category assignment and event-rate summaries.
4. Reproduce final feature list and verify exclusion of identifiers, temporal metadata, and analysis-only fields.
5. Reproduce training under the approved configuration and save validation-owned model artifacts.
6. Reproduce out-of-fold predictions and fold-level metrics.
7. Reproduce OOT test and recent predictions.
8. Reproduce ROC AUC and other agreed discrimination metrics.
9. Reproduce calibration tables and plots.
10. Reproduce risk ranking, lift/gain, and category-capture outputs.
11. Reproduce permutation feature importance and SHAP outputs.
12. Reproduce LGD integration and loss/loss-ratio outputs.
13. Compare developer and validator results at aggregate and application level.
14. Reconcile development, QA/validation, and production/pre-production outputs using identical test cases.
15. Document all discrepancies, root causes, remediation, and final status.

Acceptance criterion: Exact equality should be expected for deterministic preprocessing and identical-environment scoring where applicable. For retrained stochastic model artifacts, acceptance should be based on pre-defined tolerances for model metrics and application-level scores, unless the environment is designed to be fully deterministic.

5 Technical Model Governance

5.1 Jira Tickets (Findings) for Model Validation and SMI3

The following preliminary findings/action items are proposed from the code-based review. Severity should be finalized only after evidence and independent test results are available.

ID	Area	Preliminary Finding / Question	Status	Required Closure Evidence
SMI3-TV-01	Population	Development period is 2009–2015 while OOT testing is 2021–2023; representativeness and excluded-period rationale require evidence.	Open	Population drift study; documented rationale; sensitivity using alternative development windows if feasible.
SMI3-TV-02	Target	Business definition and horizon for target/event categories are not fully established by code alone.	Open	Approved target specification and independent category/target replication.
SMI3-TV-03	Data	Rows with missing selected fields are dropped; impact on representativeness is not yet quantified.	Open	Missingness/drop analysis and bias assessment.
SMI3-TV-04	Economic Features	Test economic variables use a December 2022 regional reference; rationale and leakage implications require confirmation.	Open	Design documentation, timing diagram, and independent leakage/sensitivity test.
SMI3-TV-05	Model Selection	Configuration restricts AutoGluon to XGBoost and disables HPO.	Open	Challenger analysis and hyperparameter/model-family rationale.
SMI3-TV-06	LGD Dependency	External LGD model is material to loss output but its validation evidence is not included.	Open	LGD approval/validation cross-reference and interface reconciliation.
SMI3-TV-07	Performance	Independent discrimination/calibration/OOT replication has not yet been completed.	Pending	Validation-owned executed results and comparison to developer results.
SMI3-TV-08	Reproducibility	Requirements generally specify minimum versions; immutable build reproducibility must be evidenced.	Open	Container digest and/or exact dependency lock evidence.
SMI3-TV-09	Deployment	Validated code-to-production artifact parity has not yet been independently demonstrated.	Pending	Commit/image/config/model checksums and environment replication.
SMI3-TV-10	Monitoring	Complete production monitoring thresholds and escalation evidence are not established by code alone.	Pending	Approved monitoring dashboard, thresholds, owners, and escalation process.
SMI3-TV-11	Governance	Privacy/fairness/proxy-variable governance is outside code evidence.	Pending	Data-governance approvals and fairness/proxy review where applicable.
SMI3-TV-12	Documentation	Formal model-development documentation must be reconciled to code.	Open	Completed documentation review and discrepancy closure.

5.2 Model Deployment

The reviewed SageMaker pipeline is designed to orchestrate preprocessing, model training, evaluation, and model registration. The inference code exposes a health endpoint, locates the packaged AutoGluon predictor, validates and preprocesses JSON input, produces model probabilities, combines outputs with loss calculations, and returns a versioned response.

Deployment validation should confirm: the same transformations are used in training/evaluation and real-time scoring; input validation rules match business requirements; error handling does not silently default material fields; model and LGD artifacts are version controlled; configuration is parameterized and environment-specific; and production output matches validation output for a controlled golden test set.

5.3 Model Monitoring Dashboard

The final production monitoring dashboard should provide a consolidated view of data quality, population stability, model performance, and operational health. The submitted code provides several evaluation artifacts but does not by itself establish the full production monitoring dashboard.

- Input completeness, invalid-value, and exception rates.
- Feature drift / PSI for major drivers.

- Score/probability distribution drift.
- Observed event rate and calibration by risk band once outcomes mature.
- ROC AUC / rank-ordering over time.
- Lift/capture at operational thresholds.
- Segment performance by region, product, LTV, and credit score.
- LGD and loss performance where applicable.
- Model/inference error rates, latency, and availability.
- Version, deployment date, and model-artifact identifiers.
- Threshold breaches, escalation status, owner, and remediation date.

6 Appendix

6.1 References / Source Files Reviewed

Primary technical sources reviewed for this draft:

- SMI2.0v3.5 LPM – Technical Validation Report (structural and reporting baseline)
- SMI3 preprocess.py, policy_processor.py, borrower_processor.py
- SMI3 train.py, evaluate.py, pipeline.py, train_config.yaml
- SMI3 predictor.py, jsonvalidator.py, jsonprocessor.py
- SMI3 lr_calculator.py, model_result.py, shap_utility.py
- SMI3 utils.py, _utils.py, and supporting CI/CD utilities
- SMI3 requirements.txt, image_utils.py, get_pipeline_definition.py, run_pipeline.py

6.2 Key Code-Evidenced Configuration

Item	Reviewed Configuration / Code Evidence
Problem type	Binary classification
Configured model family	XGBoost via AutoGluon
Evaluation metric	ROC AUC
Random seed	42
Bagging folds	5
Bagging sets	1
Stacking	0 levels in submitted train_config
Weighted ensemble	Disabled
HPO	0 trials / disabled in submitted train_config
Training period	BOOKYEAR 2009–2015; funded records
Test period	EFFECT_SID 202101–202312
Explainability	Permutation feature importance and SHAP
Tracking	MLflow
Pipeline platform	AWS SageMaker
Inference version string	SMI 3.0.5

6.3 Validation Evidence Checklist

Evidence	Required for Final Report	Current Draft Status
Approved model-development document	Yes	Not supplied / pending
Approved data lineage and data dictionary	Yes	Pending
Final development dataset snapshot	Yes	Pending independent reconciliation

Developer performance report	Yes	Pending
Validation-owned performance replication	Yes	Pending
Calibration and OOT analysis	Yes	Pending
Feature importance / SHAP review	Yes	Code implemented; executed validation pending
LGD validation cross-reference	Yes	Pending
Git commit / branch evidence	Yes	Pending
Docker image digest / dependency lock	Yes	Pending
QA / production replication	Yes	Pending
Monitoring dashboard and thresholds	Yes	Pending
Management responses to findings	Yes	Pending
Final model approval / fit-for-use decision	Yes	Not issued in this draft

6.4 Suggested Final Validation Conclusion Template

Upon completion of the independent tests, the final report should replace this paragraph with a validation-owned conclusion. The conclusion should summarize the scope completed, material findings, replicated performance, calibration, stability, implementation consistency, governance status, and any conditions of use. It should explicitly state whether the model is Fit for Use, Fit for Use with Conditions, or Not Fit for Use, and should identify any residual limitations and monitoring requirements.

This draft should be finalized only after independent execution evidence is available. Numeric performance results, final findings, severity ratings, management responses, closure status, and the final fit-for-use recommendation should be inserted from validation-owned evidence rather than inferred from source code.